Simple Dictionary

Part 2 — Screens and navigation

Phone edition • Source snapshot: 28 October 2026

How to use this document

Each source line has its own label, code and explanation. This edition includes imports, blank lines and closing brackets too.

Code is above its explanation rather than beside it. Leading indentation is removed and long code is wrapped to fit a phone. A wrapped code block is still one original source line.

Line numbers refer to the HTML walkthrough snapshot. Later source edits can change them. The existing API key is hidden in the guide only; app source is unchanged.

Start with BeginnerGuide.pdf for the overall search, favorite and preference flows. Use this document when you want to understand a particular file or line.

The file list below and PDF bookmarks help navigation. Some platform/Room/browser setup goes beyond the modules' short examples. Generated files, binary assets and Xcode project internals are outside the handwritten-source scope.

This volume: 4 files; 606 annotated source lines.

Files in reading order

1. shared/src/commonMain/kotlin/com/example/dictionary/App.kt
2. shared/src/commonMain/kotlin/com/example/dictionary/ui/InputScreen.kt
3. shared/src/commonMain/kotlin/com/example/dictionary/ui/ResultScreen.kt
4. shared/src/commonMain/kotlin/com/example/dictionary/ui/HistoryScreen.kt

1. App.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/App.kt

Connects dependencies, observes state, and opens the correct screen. It delegates networking/storage work.

Line 1

```
package com.example.dictionary
```

Places this file in the com.example.dictionary namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.compose.foundation.BorderStroke
```

Makes BorderStroke available here from androidx.compose.foundation.

Line 4

```
import androidx.compose.foundation.layout.*
```

Makes * available from androidx.compose.foundation.layout. A star imports accessible declarations from that package.

Line 5

```
import androidx.compose.material3.*
```

Makes * available from androidx.compose.material3. A star imports accessible declarations from that package.

Line 6

```
import androidx.compose.runtime.*
```

Makes * available from androidx.compose.runtime. A star imports accessible declarations from that package.

Line 7

```
import androidx.compose.ui.Modifier
```

Makes Modifier available here from androidx.compose.ui.

Line 8

```
import androidx.compose.ui.graphics.Color
```

Makes Color available here from androidx.compose.ui.graphics.

Line 9

```
import androidx.compose.ui.text.font.FontWeight
```

Makes FontWeight available here from androidx.compose.ui.text.font.

Line 10

```
import androidx.compose.ui.unit.dp
```

Makes dp available here from androidx.compose.ui.unit.

Line 11

```
import androidx.savedstate.read
```

Makes read available here from androidx.savedstate.

Line 12

```
import androidx.navigation.NavType
```

Makes NavType available here from androidx.navigation.

Line 13

```
import androidx.navigation.navArgument
```

Makes navArgument available here from androidx.navigation.

Line 14

```
import androidx.navigation.navDeepLink
```

Makes navDeepLink available here from androidx.navigation.

Line 15

```
import androidx.navigation.compose.NavHost
```

Makes NavHost available here from androidx.navigation.compose.

Line 16

```
import androidx.navigation.compose.composable
```

Makes composable available here from androidx.navigation.compose.

Line 17

```
import
androidx.navigation.compose.currentBackStackEntryAsState
```

Makes currentBackStackEntryAsState available here from androidx.navigation.compose.

Line 18

```
import androidx.navigation.compose.rememberNavController
```

Makes rememberNavController available here from androidx.navigation.compose.

Line 19

```
import com.example.dictionary.data.AppStorage
```

Makes AppStorage available here from com.example.dictionary.data.

Line 20

```
import com.example.dictionary.data.DictionaryPreferences
```

Makes DictionaryPreferences available here from com.example.dictionary.data.

Line 21

```
import
com.example.dictionary.data.DictionaryRepositoryImpl
```

Makes DictionaryRepositoryImpl available here from com.example.dictionary.data.

Line 22

```
import com.example.dictionary.data.createHttpClient
```

Makes createHttpClient available here from com.example.dictionary.data.

Line 23

```
import com.example.dictionary.domain.HistoryUseCase
```

Makes HistoryUseCase available here from com.example.dictionary.domain.

Line 24

```
import com.example.dictionary.domain.SearchWordUseCase
```

Makes SearchWordUseCase available here from com.example.dictionary.domain.

Line 25

```
import
com.example.dictionary.domain.SlangPreferenceUseCase
```

Makes SlangPreferenceUseCase available here from com.example.dictionary.domain.

Line 26

```
import
com.example.dictionary.presentation.DictionaryViewModel
```

Makes DictionaryViewModel available here from com.example.dictionary.presentation.

Line 27

```
import com.example.dictionary.presentation.UiState
```

Makes UiState available here from com.example.dictionary.presentation.

Line 28

```
import com.example.dictionary.ui.HistoryScreen
```

Makes HistoryScreen available here from com.example.dictionary.ui.

Line 29

```
import com.example.dictionary.ui.InputScreen
```

Makes InputScreen available here from com.example.dictionary.ui.

Line 30

```
import com.example.dictionary.ui.ResultScreen
```

Makes ResultScreen available here from com.example.dictionary.ui.

Line 31

```
import io.ktor.http.encodeURLParameter
```

Makes encodeURLParameter available here from io.ktor.http.

Line 32

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 33

```
// Module 8: a small, fixed set of destinations, with a
word for the result.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 34

```
sealed class Destination {
```

Defines the small set of places the app can open. Result also carries a word.

Line 35

```
object Input : Destination()
```

Declares this fixed state/destination or the values carried by this case.

Line 36

```
object History : Destination()
```

Declares this fixed state/destination or the values carried by this case.

Line 37

```
data class Result(val word: String) : Destination()
```

Defines Result as a data container.

Line 38

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 39

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 40

```
@Composable
```

Marks the following function as Compose UI code. Its body describes what to draw for the current inputs.

Line 41

```
fun App(storage: AppStorage) {
```

Defines App(). Inputs are inside parentheses; the return type describes its answer.

Line 42

```
val scope = rememberCoroutineScope()
```

Creates the coroutine scope used by this App. Compose cancels its jobs when this App leaves composition.

Line 43

```
val client = remember { createHttpClient() }
```

Creates the shared HTTP client once for this composition location, rather than once per redraw.

Line 44

```
DisposableEffect(client) {
```

Registers cleanup for this particular HTTP client.

Line 45

```
onDispose { client.close() }
```

Closes network resources when this effect is removed or its client changes.

Line 46

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 47

```
// Module 6: simple constructor injection; no DI
framework is needed.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 48

```
val viewModel = remember(storage) {
```

Keeps the state holder while this storage object stays the same. This is memory retention, not disk storage.

Line 49

```
val repository = DictionaryRepositoryImpl(
```

Builds the real repository implementation. The following arguments supply its dependencies.

Line 50

```
storage.database.historyDao(), client, "<existing API
key>"
```

Supplies database operations, the HTTP client and the existing API key. The guide hides the literal key only; the source is unchanged.

Line 51

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 52

```
DictionaryViewModel(
```

Builds the shared state holder using the three use cases and coroutine scope below.

Line 53

```
SearchWordUseCase(repository),
```

Gives the ViewModel the operation that searches through this repository.

Line 54

```
HistoryUseCase(repository),
```

Gives it history/favorite operations using the same repository.

Line 55

```
SlangPreferenceUseCase(DictionaryPreferences(storage.pref
erences)),
```

Wraps the saved preferences in their implementation and use case.

Line 56

```
scope
```

Supplies the Compose-owned coroutine scope as the final ViewModel constructor argument.

Line 57

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 58

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 59

```
val navController = rememberNavController()
```

Creates and remembers the object that tracks navigation destinations and their back stack.

Line 60

```
val backStackEntry by
navController.currentBackStackEntryAsState()
```

Observes the current navigation entry so the selected tab updates when navigation changes.

Line 61

```
val currentRoute = backStackEntry?.destination?.route ?:
"input"
```

Reads the active route; uses input before an entry is available.

Line 62

```
var searchQuery by remember { mutableStateOf("") }
```

Keeps the text currently typed in Input in Compose memory.

Line 63

```
var lastWord by remember { mutableStateOf("") }
```

Remembers the last result word for the bottom Result button; the displayed result itself reads its route argument.

Line 64

```
val state by viewModel.uiState.collectAsState()
```

Observes the result state. A change causes the parts of the screen reading it to redraw.

Line 65

```
val history by viewModel.searchHistory.collectAsState()
```

Observes the saved history list exposed by the ViewModel.

Line 66

```
val showSlang by viewModel.showSlang.collectAsState()
```

Observes the saved slang-display default exposed by the ViewModel.

Line 67

```
val snackbarHostState = remember { SnackbarHostState() }
```

Keeps the object controlling the temporary message bar.

Line 68

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 69

```
// Module 7: collect one-time messages separately from
persistent screen state.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 70

```
LaunchedEffect(viewModel) {
```

Starts one event collector for this ViewModel; it is cancelled if the effect leaves composition.

Line 71

```
viewModel.events.collect { message ->
snackbarHostState.showSnackbar(message) }
```

Waits for one-time messages and shows each one in a snackbar.

Line 72

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 73

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 74

```
fun navigate(destination: Destination) {
```

Defines App’s helper for changing screens.

Line 75

```
val route = when (destination) {
```

Selects the route string based on the kind of destination.

Line 76

```
Destination.Input -> "input"
```

The Input destination uses the literal route input.

Line 77

```
Destination.History -> "history"
```

The History destination uses the literal route history.

Line 78

```
is Destination.Result -> {
```

For a result destination, its word must be included in the route.

Line 79

```
val word = destination.word.trim().lowercase()
```

Normalizes the word before putting it in a navigation route.

Line 80

```
if (word.isBlank()) "result" else
"result/${word.encodeURLParameter()}"
```

Uses an empty-result route for blank input; otherwise URL-encodes the word so route punctuation is handled safely.

Line 81

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 82

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 83

```
navController.navigate(route) {
```

Asks the controller to open the chosen route.

Line 84

```
launchSingleTop = true
```

Avoids adding another copy when this destination is already at the top. It does not remove older different entries.

Line 85

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 86

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 87

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 88

```
MaterialTheme(
```

Supplies the colors/styles that Material components read while drawing.

Line 89

```
colorScheme = lightColorScheme(
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 90

```
primary = Color.Black,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 91

```
onPrimary = Color.White,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 92

```
surface = Color.White,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 93

```
onSurface = Color.Black,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 94

```
surfaceVariant = Color(0xFFF4F4F6)
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 95

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 96

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 97

```
Scaffold(
```

Creates the page frame, including the message area, bottom navigation and screen content.

Line 98

```
snackbarHost = { SnackbarHost(snackbarHostState) },
```

Places the message bar connected to snackbarHostState in the Scaffold.

Line 99

```
bottomBar = {
```

Starts the content shown at the bottom of the app.

Line 100

```
Surface(
```

A visual container providing background, border and shape.

Line 101

```
modifier = Modifier.fillMaxWidth(),
```

Applies the following layout/appearance instructions to this UI element.

Line 102

```
color = Color.White,
```

Sets this element’s color.

Line 103

```
border = BorderStroke(1.dp, Color.Black)
```

Sets the outline width and color.

Line 104

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 105

```
NavigationBar(containerColor = Color.White,
tonalElevation = 0.dp) {
```

Draws the bottom-navigation container.

Line 106

```
NavigationBarItem(
```

Draws one bottom-navigation choice.

Line 107

```
icon = { Text("01", fontWeight = FontWeight.Bold) },
```

Defines the icon content; here the item can use text instead of an image.

Line 108

```
label = { Text("INPUT") },
```

Defines the label content of a navigation item.

Line 109

```
selected = currentRoute == "input",
```

Marks this bottom item selected when its route matches the current screen.

Line 110

```
onClick = { navigate(Destination.Input) }
```

Makes tapping this bottom item open its associated destination.

Line 111

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 112

```
NavigationBarItem(
```

Draws one bottom-navigation choice.

Line 113

```
icon = { Text("02", fontWeight = FontWeight.Bold) },
```

Defines the icon content; here the item can use text instead of an image.

Line 114

```
label = { Text("RESULT") },
```

Defines the label content of a navigation item.

Line 115

```
selected = currentRoute.startsWith("result"),
```

Marks this bottom item selected when its route matches the current screen.

Line 116

```
onClick = { navigate(Destination.Result(lastWord)) }
```

Makes tapping this bottom item open its associated destination.

Line 117

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 118

```
NavigationBarItem(
```

Draws one bottom-navigation choice.

Line 119

```
icon = { Text("03", fontWeight = FontWeight.Bold) },
```

Defines the icon content; here the item can use text instead of an image.

Line 120

```
label = { Text("HISTORY") },
```

Defines the label content of a navigation item.

Line 121

```
selected = currentRoute == "history",
```

Marks this bottom item selected when its route matches the current screen.

Line 122

```
onClick = { navigate(Destination.History) }
```

Makes tapping this bottom item open its associated destination.

Line 123

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 124

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 125

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 126

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 127

```
) { innerPadding ->
```

Receives the padding needed to avoid drawing screen content under Scaffold bars.

Line 128

```
NavHost(
```

Connects a navigation controller to a group of screen definitions.

Line 129

```
navController = navController,
```

Tells NavHost which remembered navigation controller to use.

Line 130

```
startDestination = "input",
```

Opens Input when the graph first starts without another destination request.

Line 131

```
modifier = Modifier.fillMaxSize().padding(innerPadding)
```

Applies the following layout/appearance instructions to this UI element.

Line 132

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 133

```
composable("input") {
```

Defines the Input screen destination.

Line 134

```
InputScreen(
```

Calls the shared input UI with the supplied values and action callbacks.

Line 135

```
query = searchQuery,
```

Supplies the current text value; InputScreen does not own a second independent query.

Line 136

```
onQueryChange = { searchQuery = it },
```

When typing changes, store the new text in App’s searchQuery.

Line 137

```
onSearchClick = { navigate(Destination.Result(it)) },
```

When Input reports a search, navigate to that word’s result route.

Line 138

```
showSlang = showSlang,
```

Supplies the saved slang-default value to this part of the UI.

Line 139

```
onShowSlangChange = { viewModel.saveShowSlang(it) }
```

Forward the checkbox value to the ViewModel, which saves it through the use case.

Line 140

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 141

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 142

```
composable("result") {
```

Defines the result tab when no word has been selected.

Line 143

```
ResultScreen("", UiState.Idle, showSlang, {}, {})
```

Shows the initial result screen with no word and no-op action callbacks.

Line 144

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 145

```
// Module 8: pass the word through the route, including
deep links.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 146

```
composable(
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 147

```
route = "result/{word}",
```

Declares a route with a required word placeholder.

Line 148

```
arguments = listOf(navArgument("word") { type =
NavType.StringType }),
```

Specifies that the word route argument is a String.

Line 149

```
deepLinks = listOf(navDeepLink { uriPattern =
"dictionary://word/{word}" })
```

Associates this destination with dictionary://word/{word}. OS URL registration is a separate platform step.

Line 150

```
) { entry ->
```

Starts a branch/callback and gives the incoming value the name shown before the arrow.

Line 151

```
val word = entry.arguments?.read { getString("word")
}.orEmpty()
```

Reads the selected word using the SavedState API expected by the current Navigation library; null arguments become an empty string.

Line 152

```
LaunchedEffect(word) {
```

Runs the lookup effect when this destination enters composition, or when its word changes.

Line 153

```
lastWord = word
```

Updates the word used by the bottom Result tab.

Line 154

```
searchQuery = word
```

Also puts the selected word into Input’s text value.

Line 155

```
viewModel.searchWord(word)
```

Starts the result lookup through the state holder.

Line 156

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 157

```
ResultScreen(
```

Calls the shared result UI with its word, state and action callbacks.

Line 158

```
word = word,
```

Passes the relevant word to the object/component being built.

Line 159

```
state = state,
```

Passes the current sealed result state to the UI.

Line 160

```
showSlangByDefault = showSlang,
```

Supplies the saved slang-default value to this part of the UI.

Line 161

```
onRefresh = { viewModel.searchWord(word, forceRefresh =
true) },
```

Requests a new lookup that skips the initial saved-word lookup.

Line 162

```
onSearchSuggestion = { navigate(Destination.Result(it)) }
```

Tapping a suggestion navigates to that suggested word.

Line 163

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 164

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 165

```
composable("history") {
```

Defines the History screen destination.

Line 166

```
HistoryScreen(
```

Calls the shared saved-history UI with its list and callbacks.

Line 167

```
history = history,
```

Passes the observed history list into its screen.

Line 168

```
onSelectWord = { navigate(Destination.Result(it)) },
```

Opening a history row navigates to the selected word; normal lookup can return its saved definition.

Line 169

```
onToggleFavorite = { viewModel.toggleFavorite(it) }
```

Forward the star action to the ViewModel instead of modifying the row in the screen.

Line 170

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 171

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 172

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 173

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 174

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 175

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

2. InputScreen.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/ui/InputScreen.kt

Draws search input/settings and reports user actions via callbacks.

Line 1

```
package com.example.dictionary.ui
```

Places this file in the com.example.dictionary.ui namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.compose.foundation.BorderStroke
```

Makes BorderStroke available here from androidx.compose.foundation.

Line 4

```
import androidx.compose.foundation.layout.*
```

Makes * available from androidx.compose.foundation.layout. A star imports accessible declarations from that package.

Line 5

```
import androidx.compose.material3.*
```

Makes * available from androidx.compose.material3. A star imports accessible declarations from that package.

Line 6

```
import androidx.compose.runtime.*
```

Makes * available from androidx.compose.runtime. A star imports accessible declarations from that package.

Line 7

```
import androidx.compose.ui.Alignment
```

Makes Alignment available here from androidx.compose.ui.

Line 8

```
import androidx.compose.ui.Modifier
```

Makes Modifier available here from androidx.compose.ui.

Line 9

```
import androidx.compose.ui.graphics.Color
```

Makes Color available here from androidx.compose.ui.graphics.

Line 10

```
import androidx.compose.ui.text.font.FontFamily
```

Makes FontFamily available here from androidx.compose.ui.text.font.

Line 11

```
import androidx.compose.ui.text.font.FontWeight
```

Makes FontWeight available here from androidx.compose.ui.text.font.

Line 12

```
import androidx.compose.ui.unit.dp
```

Makes dp available here from androidx.compose.ui.unit.

Line 13

```
import androidx.compose.ui.unit.sp
```

Makes sp available here from androidx.compose.ui.unit.

Line 14

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 15

```
@Composable
```

Marks the following function as Compose UI code. Its body describes what to draw for the current inputs.

Line 16

```
fun InputScreen(
```

Defines InputScreen(). Inputs are inside parentheses; the return type describes its answer.

Line 17

```
query: String,
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 18

```
onQueryChange: (String) -> Unit,
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 19

```
onSearchClick: (String) -> Unit,
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 20

```
showSlang: Boolean,
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 21

```
onShowSlangChange: (Boolean) -> Unit
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 22

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 23

```
Surface(modifier = Modifier.fillMaxSize(), color =
Color(0xFFF9F9FB)) {
```

A visual container providing background, border and shape.

Line 24

```
Column(
```

A vertical layout: the children below are arranged top to bottom.

Line 25

```
modifier = Modifier.fillMaxSize().padding(32.dp),
```

Applies the following layout/appearance instructions to this UI element.

Line 26

```
horizontalAlignment = Alignment.Start,
```

Controls horizontal alignment of children.

Line 27

```
verticalArrangement = Arrangement.Center
```

Controls how children are spread vertically.

Line 28

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 29

```
Text(
```

Draws text using the supplied value/style.

Line 30

```
text = "// KMP Dictionary",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 31

```
style = MaterialTheme.typography.labelLarge.copy(
```

Chooses a theme text style or a customized copy of it.

Line 32

```
letterSpacing = 2.sp,
```

Sets the spacing between letters.

Line 33

```
fontWeight = FontWeight.Bold
```

Sets how bold the text is.

Line 34

```
),
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 35

```
color = Color.Black
```

Sets this element’s color.

Line 36

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 37

```
Spacer(modifier = Modifier.height(12.dp))
```

Adds empty space between UI elements.

Line 38

```
Text(
```

Draws text using the supplied value/style.

Line 39

```
text = "Look up definitions & slang.",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 40

```
style = MaterialTheme.typography.headlineLarge.copy(
```

Chooses a theme text style or a customized copy of it.

Line 41

```
fontWeight = FontWeight.ExtraBold,
```

Sets how bold the text is.

Line 42

```
fontSize = 32.sp
```

Sets text size using sp, which respects font scaling.

Line 43

```
),
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 44

```
color = Color.Black
```

Sets this element's color.

Line 45

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 46

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 47

```
Spacer(modifier = Modifier.height(40.dp))
```

Adds empty space between UI elements.

Line 48

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 49

```
OutlinedTextField(
```

Draws the editable word input with an outline.

Line 50

```
value = query,
```

Draw the current query supplied by App.

Line 51

```
onValueChange = onQueryChange,
```

Send newly typed text to App rather than keeping a separate local query.

Line 52

```
placeholder = { Text("enter target word...", color =
Color.Gray) },
```

Defines the hint shown while the text field is empty.

Line 53

```
modifier = Modifier.fillMaxWidth(),
```

Applies the following layout/appearance instructions to this UI element.

Line 54

```
singleLine = true,
```

Keeps the input to a single line.

Line 55

```
shape = Shapes().extraSmall,
```

Chooses the component’s corner shape.

Line 56

```
colors = OutlinedTextFieldDefaults.colors(
```

Supplies the Material component color configuration.

Line 57

```
focusedBorderColor = Color.Black,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 58

```
unfocusedBorderColor = Color.DarkGray,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 59

```
focusedContainerColor = Color.White,
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 60

```
unfocusedContainerColor = Color.White
```

Supplies this named value to the surrounding function/configuration. Follow the containing call above to see who receives it.

Line 61

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 62

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 63

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 64

```
Row(verticalAlignment = Alignment.CenterVertically) {
```

A horizontal layout: the children below are arranged side by side.

Line 65

```
Checkbox(checked = showSlang, onCheckedChange =
onShowSlangChange)
```

Draw the saved slang-default value; send changes through the supplied preference callback.

Line 66

```
Text("Show slang by default (may contain explicit
language)",
```

Draws text using the supplied value/style.

Line 67

```
style = MaterialTheme.typography.bodySmall)
```

Chooses a theme text style or a customized copy of it.

Line 68

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 69

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 70

```
Spacer(modifier = Modifier.height(16.dp))
```

Adds empty space between UI elements.

Line 71

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 72

```
Button(
```

Draws a clickable button and connects the action supplied below.

Line 73

```
onClick = { if (query.isNotBlank()) onSearchClick(query)
},
```

When Search is tapped, ignore blank input and send the query to App’s callback.

Line 74

```
modifier = Modifier.fillMaxWidth().height(52.dp),
```

Applies the following layout/appearance instructions to this UI element.

Line 75

```
shape = Shapes().extraSmall,
```

Chooses the component’s corner shape.

Line 76

```
colors = ButtonDefaults.buttonColors(containerColor =
Color.Black)
```

Supplies the Material component color configuration.

Line 77

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 78

```
Text(
```

Draws text using the supplied value/style.

Line 79

```
text = "EXECUTE SEARCH →",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 80

```
style = MaterialTheme.typography.titleSmall.copy(
```

Chooses a theme text style or a customized copy of it.

Line 81

```
letterSpacing = 1.5.sp,
```

Sets the spacing between letters.

Line 82

```
fontWeight = FontWeight.Bold
```

Sets how bold the text is.

Line 83

```
),
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 84

```
color = Color.White
```

Sets this element’s color.

Line 85

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 86

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 87

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 88

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 89

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

3. ResultScreen.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/ui/ResultScreen.kt

Draws the result state and definition cards. Slang show/hide is local UI state.

Line 1

```
package com.example.dictionary.ui
```

Places this file in the com.example.dictionary.ui namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.compose.foundation.BorderStroke
```

Makes BorderStroke available here from androidx.compose.foundation.

Line 4

```
import androidx.compose.foundation.background
```

Makes background available here from androidx.compose.foundation.

Line 5

```
import androidx.compose.foundation.clickable
```

Makes clickable available here from androidx.compose.foundation.

Line 6

```
import androidx.compose.foundation.layout.*
```

Makes * available from androidx.compose.foundation.layout. A star imports accessible declarations from that package.

Line 7

```
import androidx.compose.foundation.lazy.LazyColumn
```

Makes LazyColumn available here from androidx.compose.foundation.lazy.

Line 8

```
import androidx.compose.foundation.lazy.items
```

Makes items available here from androidx.compose.foundation.lazy.

Line 9

```
import androidx.compose.material3.*
```

Makes * available from androidx.compose.material3. A star imports accessible declarations from that package.

Line 10

```
import androidx.compose.runtime.*
```

Makes * available from androidx.compose.runtime. A star imports accessible declarations from that package.

Line 11

```
import androidx.compose.ui.Alignment
```

Makes Alignment available here from androidx.compose.ui.

Line 12

```
import androidx.compose.ui.Modifier
```

Makes Modifier available here from androidx.compose.ui.

Line 13

```
import androidx.compose.ui.graphics.Color
```

Makes Color available here from androidx.compose.ui.graphics.

Line 14

```
import androidx.compose.ui.text.font.FontWeight
```

Makes FontWeight available here from androidx.compose.ui.text.font.

Line 15

```
import androidx.compose.ui.unit.dp
```

Makes dp available here from androidx.compose.ui.unit.

Line 16

```
import androidx.compose.ui.unit.sp
```

Makes sp available here from androidx.compose.ui.unit.

Line 17

```
import com.example.dictionary.domain.DictionaryEntry
```

Makes DictionaryEntry available here from com.example.dictionary.domain.

Line 18

```
import com.example.dictionary.presentation.UiState
```

Makes UiState available here from com.example.dictionary.presentation.

Line 19

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 20

```
@Composable
```

Marks the following function as Compose UI code. Its body describes what to draw for the current inputs.

Line 21

```
fun ResultScreen(
```

Defines ResultScreen(). Inputs are inside parentheses; the return type describes its answer.

Line 22

```
word: String,
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 23

```
state: UiState,
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 24

```
showSlangByDefault: Boolean,
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 25

```
onRefresh: () -> Unit,
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 26

```
onSearchSuggestion: (String) -> Unit
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 27

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 28

```
var showUrbanSlang by remember(word, showSlangByDefault)
{ mutableStateOf(showSlangByDefault) }
```

Keeps the current result's expansion locally. Initialize from the saved default when the word/default changes; a local expand/collapse is not a preference write.

Line 29

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 30

```
Surface(modifier = Modifier.fillMaxSize(), color =
Color(0xFFF9F9FB)) {
```

A visual container providing background, border and shape.

Line 31

```
Column(modifier = Modifier.fillMaxSize().padding(24.dp))
{
```

A vertical layout: the children below are arranged top to bottom.

Line 32

```
if (word.isNotBlank()) {
```

Checks this condition and runs the following branch only when it is true: (word.isNotBlank()).

Line 33

```
Row(
```

A horizontal layout: the children below are arranged side by side.

Line 34

```
modifier = Modifier.fillMaxWidth(),
```

Applies the following layout/appearance instructions to this UI element.

Line 35

```
horizontalArrangement = Arrangement.SpaceBetween,
```

Controls how children are spread horizontally.

Line 36

```
verticalAlignment = Alignment.Bottom
```

Controls vertical alignment of children.

Line 37

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 38

```
Text(
```

Draws text using the supplied value/style.

Line 39

```
text = word.uppercase(),
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 40

```
style = MaterialTheme.typography.headlineMedium.copy(
```

Chooses a theme text style or a customized copy of it.

Line 41

```
fontWeight = FontWeight.Black,
```

Sets how bold the text is.

Line 42

```
letterSpacing = 1.sp
```

Sets the spacing between letters.

Line 43

```
),
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 44

```
color = Color.Black
```

Sets this element’s color.

Line 45

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 46

```
Text(
```

Draws text using the supplied value/style.

Line 47

```
text = "QUERY_RESOLVED",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 48

```
style = MaterialTheme.typography.labelSmall,
```

Chooses a theme text style or a customized copy of it.

Line 49

```
color = Color.Gray
```

Sets this element’s color.

Line 50

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 51

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 52

```
Spacer(modifier = Modifier.height(8.dp))
```

Adds empty space between UI elements.

Line 53

```
HorizontalDivider(color = Color.Black, thickness = 2.dp)
```

Draws a dividing line.

Line 54

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 55

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 56

```
if (word.isNotBlank()) {
```

Checks this condition and runs the following branch only when it is true: (word.isNotBlank()).

Line 57

```
TextButton(onClick = onRefresh, enabled = state !is
UiState.Loading) {
```

Draws a spinner while waiting for a search.

Line 58
```
Text("Refresh definitions")
```
Draws text using the supplied value/style.

Line 59
```
}
```
Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 60
```
}
```
Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 61
```
Spacer(modifier = Modifier.height(16.dp))
```
Adds empty space between UI elements.

Line 62
```
[blank line]
```
Blank line: separates related code visually; it performs no operation.

Line 63
```
Box(modifier = Modifier.fillMaxSize()) {
```
A container that can position/stack its children.

Line 64
```
when (state) {
```
Selects the UI for the complete result state family.

Line 65
```
is UiState.Loading -> CircularProgressIndicator(
```
Draws a spinner while waiting for a search.

Line 66
```
modifier = Modifier.align(Alignment.Center),
```
Applies the following layout/appearance instructions to this UI element.

Line 67

```
color = Color.Black
```

Sets this element’s color.

Line 68

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 69

```
is UiState.Error -> Text(
```

Displays the readable error carried by this state.

Line 70

```
text = "ERROR: ${state.message}",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 71

```
color = Color.Red,
```

Sets this element’s color.

Line 72

```
modifier = Modifier.align(Alignment.Center)
```

Applies the following layout/appearance instructions to this UI element.

Line 73

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 74

```
is UiState.Success -> {
```

Starts the list that displays available definitions and suggestions.

Line 75

```
LazyColumn(
```

Draws a scrollable vertical list that creates items as needed.

Line 76

```
modifier = Modifier.fillMaxSize(),
```

Applies the following layout/appearance instructions to this UI element.

Line 77

```
verticalArrangement = Arrangement.spacedBy(12.dp)
```

Controls how children are spread vertically.

Line 78

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 79

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 80

```
// Render "Did you mean...?" banner if Merriam-Webster
offered a correction
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 81

```
if (!state.didYouMeanSuggestion.isNullOrBlank()) {
```

Checks this condition and runs the following branch only when it is true: (! state.didYouMeanSuggestion.isNullOrBlank()).

Line 82

```
item {
```

Adds one non-repeating item to this LazyColumn, such as a heading, suggestion banner or slang section.

Line 83

```
Surface(
```

A visual container providing background, border and shape.

Line 84

```
modifier = Modifier
```

Applies the following layout/appearance instructions to this UI element.

Line 85

```
.fillMaxWidth()
```

Uses the maximum available width in the parent layout.

Line 86

```
.clickable {
onSearchSuggestion(state.didYouMeanSuggestion) },
```

Reports the suggested word to App so it can navigate to that result.

Line 87

```
color = Color(0xFFE3F2FD),
```

Sets this element's color.

Line 88

```
border = BorderStroke(1.dp, Color(0xFF90CAF9)),
```

Sets the outline width and color.

Line 89

```
shape = Shapes().extraSmall
```

Chooses the component's corner shape.

Line 90

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 91

```
Row(
```

A horizontal layout: the children below are arranged side by side.

Line 92

```
modifier = Modifier.padding(14.dp),
```

Applies the following layout/appearance instructions to this UI element.

Line 93

```
verticalAlignment = Alignment.CenterVertically
```

Controls vertical alignment of children.

Line 94

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 95

```
Text("💡", fontSize = 16.sp)
```

Draws text using the supplied value/style.

Line 96

```
Spacer(modifier = Modifier.width(12.dp))
```

Adds empty space between UI elements.

Line 97

```
Column {
```

Starts a vertical layout using its default configuration; the next children are drawn from top to bottom.

Line 98

```
Text(
```

Draws text using the supplied value/style.

Line 99

```
"Did you mean:
${state.didYouMeanSuggestion.uppercase()}?",
```

A text/String value used by the surrounding call, query or configuration. Its role is determined by that surrounding operation.

Line 100

```
style =
MaterialTheme.typography.labelLarge.copy(fontWeight =
FontWeight.Bold),
```

Chooses a theme text style or a customized copy of it.

Line 101

```
color = Color(0xFF0D47A1)
```

Sets this element's color.

Line 102

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 103

```
Text(
```

Draws text using the supplied value/style.

Line 104

```
"Click here to search the corrected spelling.",
```

A text/String value used by the surrounding call, query or configuration. Its role is determined by that surrounding operation.

Line 105

```
style = MaterialTheme.typography.bodySmall,
```

Chooses a theme text style or a customized copy of it.

Line 106

```
color = Color(0xFF1565C0)
```

Sets this element's color.

Line 107

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 108

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 109

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 110

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 111

```
Spacer(modifier = Modifier.height(4.dp))
```

Adds empty space between UI elements.

Line 112

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 113

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 114

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 115

```
if (state.formalEntries.isNotEmpty()) {
```

Checks this condition and runs the following branch only when it is true: (state.formalEntries.isNotEmpty()).

Line 116

```
item {
```

Adds one non-repeating item to this LazyColumn, such as a heading, suggestion banner or slang section.

Line 117

```
Text(
```

Draws text using the supplied value/style.

Line 118

```
"// FORMAL DEFINITIONS",
```

A text/String value used by the surrounding call, query or configuration. Its role is determined by that surrounding operation.

Line 119

```
style =
MaterialTheme.typography.labelMedium.copy(fontWeight =
FontWeight.Bold),
```

Chooses a theme text style or a customized copy of it.

Line 120

```
color = Color.DarkGray
```

Sets this element’s color.

Line 121

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 122

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 123

```
items(state.formalEntries) { entry ->
```

Creates one formal definition card for each formal entry.

Line 124

```
SharpEntryCard(entry, isSlang = false)
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 125

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 126

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 127

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 128

```
if (state.urbanEntries.isNotEmpty()) {
```

Checks this condition and runs the following branch only when it is true: (state.urbanEntries.isNotEmpty()).

Line 129

```
item {
```

Adds one non-repeating item to this LazyColumn, such as a heading, suggestion banner or slang section.

Line 130

```
Spacer(modifier = Modifier.height(8.dp))
```

Adds empty space between UI elements.

Line 131

```
Surface(
```

A visual container providing background, border and shape.

Line 132

```
modifier = Modifier
```

Applies the following layout/appearance instructions to this UI element.

Line 133

```
.fillMaxWidth()
```

Uses the maximum available width in the parent layout.

Line 134

```
.clickable { showUrbanSlang = !showUrbanSlang },
```

Expands/collapses slang locally on this result; it does not overwrite the saved default.

Line 135

```
color = Color(0xFFFFF3CD),
```

Sets this element's color.

Line 136

```
border = BorderStroke(1.dp, Color(0xFFFFEEBA)),
```

Sets the outline width and color.

Line 137

```
shape = Shapes().extraSmall
```

Chooses the component's corner shape.

Line 138

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 139

```
Row(
```

A horizontal layout: the children below are arranged side by side.

Line 140

```
modifier = Modifier.padding(16.dp),
```

Applies the following layout/appearance instructions to this UI element.

Line 141

```
verticalAlignment = Alignment.CenterVertically
```

Controls vertical alignment of children.

Line 142

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 143

```
Text("⚠️", fontSize = 16.sp)
```

Draws text using the supplied value/style.

Line 144

```
Spacer(modifier = Modifier.width(12.dp))
```

Adds empty space between UI elements.

Line 145

```
Column(modifier = Modifier.weight(1f)) {
```

A vertical layout: the children below are arranged top to bottom.

Line 146

```
Text(
```

Draws text using the supplied value/style.

Line 147

```
"URBAN SLANG MATRIX",
```

A text/String value used by the surrounding call, query or configuration. Its role is determined by that surrounding operation.

Line 148

```
style =
MaterialTheme.typography.labelLarge.copy(fontWeight =
FontWeight.Bold),
```

Chooses a theme text style or a customized copy of it.

Line 149

```
color = Color(0xFF856404)
```

Sets this element's color.

Line 150

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 151

```
Text(
```

Draws text using the supplied value/style.

Line 152

```
if (showUrbanSlang) "Click to collapse unstructured
dataset" else "Contains unfiltered explicit terms. Click
to expand.",
```

Checks this condition and runs the following branch only when it is true: (showUrbanSlang) "Click to collapse unstructured dataset" else "Contains unfiltered explicit terms. Click to expand.",.

Line 153

```
style = MaterialTheme.typography.bodySmall,
```

Chooses a theme text style or a customized copy of it.

Line 154

```
color = Color(0xFF856404)
```

Sets this element’s color.

Line 155

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 156

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 157

```
Text(if (showUrbanSlang) "[HIDE]" else "[SHOW]",
fontWeight = FontWeight.Bold, color = Color(0xFF856404))
```

Draws text using the supplied value/style.

Line 158

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 159

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 160

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 161

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 162

```
item {
```

Adds one non-repeating item to this LazyColumn, such as a heading, suggestion banner or slang section.

Line 163

```
Column(
```

A vertical layout: the children below are arranged top to bottom.

Line 164

```
modifier = Modifier.fillMaxWidth()
```

Applies the following layout/appearance instructions to this UI element.

Line 165

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 166

```
if (showUrbanSlang) {
```

Checks this condition and runs the following branch only when it is true: (showUrbanSlang).

Line 167

```
Spacer(modifier = Modifier.height(12.dp))
```

Adds empty space between UI elements.

Line 168

```
Text(
```

Draws text using the supplied value/style.

Line 169

```
"// SLANG ENTRIES",
```

A text/String value used by the surrounding call, query or configuration. Its role is determined by that surrounding operation.

Line 170

```
style =
MaterialTheme.typography.labelMedium.copy(fontWeight =
FontWeight.Bold),
```

Chooses a theme text style or a customized copy of it.

Line 171

```
color = Color.DarkGray
```

Sets this element's color.

Line 172

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 173

```
Spacer(modifier = Modifier.height(8.dp))
```

Adds empty space between UI elements.

Line 174

```
Column(verticalArrangement = Arrangement.spacedBy(12.dp))
 {
```

A vertical layout: the children below are arranged top to bottom.

Line 175

```
state.urbanEntries.forEach { entry ->
```

Creates a card for each slang entry when that section is expanded.

Line 176

```
SharpEntryCard(entry, isSlang = true)
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 177

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 178

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 179

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 180

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 181

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 182

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 183

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 184

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 185

```
is UiState.Idle -> Text(
```

Displays the initial guidance before a word result exists.

Line 186

```
"No active query. Input a target term.",
```

A text/String value used by the surrounding call, query or configuration. Its role is determined by that surrounding operation.

Line 187

```
modifier = Modifier.align(Alignment.Center),
```

Applies the following layout/appearance instructions to this UI element.

Line 188

```
color = Color.Gray
```

Sets this element’s color.

Line 189

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 190

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 191

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 192

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 193

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 194

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 195

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 196

```
@Composable
```

Marks the following function as Compose UI code. Its body describes what to draw for the current inputs.

Line 197

```
fun SharpEntryCard(entry: DictionaryEntry, isSlang:
Boolean) {
```

Defines the reusable visual card for one DictionaryEntry.

Line 198

```
Surface(
```

A visual container providing background, border and shape.

Line 199

```
modifier = Modifier.fillMaxWidth(),
```

Applies the following layout/appearance instructions to this UI element.

Line 200

```
color = Color.White,
```

Sets this element’s color.

Line 201

```
border = BorderStroke(1.dp, Color.Black),
```

Sets the outline width and color.

Line 202

```
shape = Shapes().extraSmall
```

Chooses the component’s corner shape.

Line 203

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 204

```
Column(modifier = Modifier.padding(16.dp)) {
```

A vertical layout: the children below are arranged top to bottom.

Line 205

```
Row(
```

A horizontal layout: the children below are arranged side by side.

Line 206

```
modifier = Modifier.fillMaxWidth(),
```

Applies the following layout/appearance instructions to this UI element.

Line 207

```
horizontalArrangement = Arrangement.SpaceBetween,
```

Controls how children are spread horizontally.

Line 208

```
verticalAlignment = Alignment.CenterVertically
```

Controls vertical alignment of children.

Line 209

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 210

```
Text(
```

Draws text using the supplied value/style.

Line 211

```
text = "[ ${entry.source.uppercase()} ]",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 212

```
style =
MaterialTheme.typography.labelSmall.copy(fontWeight =
FontWeight.Bold),
```

Chooses a theme text style or a customized copy of it.

Line 213

```
color = if (isSlang) Color(0xFFD32F2F) else Color.Black
```

Sets this element’s color.

Line 214

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 215

```
if (entry.partOfSpeech.isNotEmpty()) {
```

Checks this condition and runs the following branch only when it is true: (entry.partOfSpeech.isNotEmpty()).

Line 216

```
Text(
```

Draws text using the supplied value/style.

Line 217

```
text = entry.partOfSpeech,
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 218

```
style = MaterialTheme.typography.labelSmall,
```

Chooses a theme text style or a customized copy of it.

Line 219

```
color = Color.Gray
```

Sets this element’s color.

Line 220

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 221

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 222

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 223

```
Spacer(modifier = Modifier.height(8.dp))
```

Adds empty space between UI elements.

Line 224

```
Text(
```

Draws text using the supplied value/style.

Line 225

```
text = entry.definition,
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 226

```
style =
MaterialTheme.typography.bodyMedium.copy(fontWeight =
FontWeight.Normal),
```

Chooses a theme text style or a customized copy of it.

Line 227

```
color = Color.Black
```

Sets this element's color.

Line 228

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 229

```
if (entry.example.isNotEmpty()) {
```

Checks this condition and runs the following branch only when it is true: (entry.example.isNotEmpty()).

Line 230

```
Spacer(modifier = Modifier.height(8.dp))
```

Adds empty space between UI elements.

Line 231

```
Box(
```

A container that can position/stack its children.

Line 232

```
modifier = Modifier
```

Applies the following layout/appearance instructions to this UI element.

Line 233

```
.fillMaxWidth()
```

Uses the maximum available width in the parent layout.

Line 234

```
.background(Color(0xFFF4F4F6))
```

Paints the specified background behind this component.

Line 235

```
.padding(10.dp)
```

Adds spacing inside/around this component using the specified dp values.

Line 236

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 237

```
Text(
```

Draws text using the supplied value/style.

Line 238

```
text = "EX: \"${entry.example}\"",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 239

```
style = MaterialTheme.typography.bodySmall,
```

Chooses a theme text style or a customized copy of it.

Line 240

```
color = Color.DarkGray
```

Sets this element’s color.

Line 241

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 242

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 243

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 244

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 245

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 246

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

4. HistoryScreen.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/ui/HistoryScreen.kt

Draws already-observed history and reports row/star taps.

Line 1

```
package com.example.dictionary.ui
```

Places this file in the com.example.dictionary.ui namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.compose.foundation.BorderStroke
```

Makes BorderStroke available here from androidx.compose.foundation.

Line 4

```
import androidx.compose.foundation.clickable
```

Makes clickable available here from androidx.compose.foundation.

Line 5

```
import androidx.compose.foundation.layout.*
```

Makes * available from androidx.compose.foundation.layout. A star imports accessible declarations from that package.

Line 6

```
import androidx.compose.foundation.lazy.LazyColumn
```

Makes LazyColumn available here from androidx.compose.foundation.lazy.

Line 7

```
import androidx.compose.foundation.lazy.items
```

Makes items available here from androidx.compose.foundation.lazy.

Line 8

```
import androidx.compose.material3.*
```

Makes * available from androidx.compose.material3. A star imports accessible declarations from that package.

Line 9

```
import androidx.compose.runtime.*
```

Makes * available from androidx.compose.runtime. A star imports accessible declarations from that package.

Line 10

```
import androidx.compose.ui.Alignment
```

Makes Alignment available here from androidx.compose.ui.

Line 11

```
import androidx.compose.ui.Modifier
```

Makes Modifier available here from androidx.compose.ui.

Line 12

```
import androidx.compose.ui.graphics.Color
```

Makes Color available here from androidx.compose.ui.graphics.

Line 13

```
import androidx.compose.ui.text.font.FontWeight
```

Makes FontWeight available here from androidx.compose.ui.text.font.

Line 14

```
import androidx.compose.ui.unit.dp
```

Makes dp available here from androidx.compose.ui.unit.

Line 15

```
import com.example.dictionary.domain.HistoryItem
```

Makes HistoryItem available here from com.example.dictionary.domain.

Line 16

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 17

```
@Composable
```

Marks the following function as Compose UI code. Its body describes what to draw for the current inputs.

Line 18

```
fun HistoryScreen(
```

Defines HistoryScreen(). Inputs are inside parentheses; the return type describes its answer.

Line 19

```
history: List<HistoryItem>,
```

Declares an input parameter and its type. The screen/function uses the value supplied by its caller.

Line 20

```
onSelectWord: (String) -> Unit,
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 21

```
onToggleFavorite: (String) -> Unit
```

Callback parameter: the caller supplies the action. This screen reports the event rather than implementing the storage/network operation here.

Line 22

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 23

```
Surface(modifier = Modifier.fillMaxSize(), color =
Color(0xFFF9F9FB)) {
```

A visual container providing background, border and shape.

Line 24

```
Column(modifier = Modifier.fillMaxSize().padding(24.dp))
{
```

A vertical layout: the children below are arranged top to bottom.

Line 25

```
Row(
```

A horizontal layout: the children below are arranged side by side.

Line 26

```
modifier = Modifier.fillMaxWidth(),
```

Applies the following layout/appearance instructions to this UI element.

Line 27

```
horizontalArrangement = Arrangement.SpaceBetween,
```

Controls how children are spread horizontally.

Line 28

```
verticalAlignment = Alignment.Bottom
```

Controls vertical alignment of children.

Line 29

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 30

```
Text(
```

Draws text using the supplied value/style.

Line 31

```
text = "SEARCH_LEDGER",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 32

```
style =
MaterialTheme.typography.headlineSmall.copy(fontWeight =
FontWeight.Black),
```

Chooses a theme text style or a customized copy of it.

Line 33

```
color = Color.Black
```

Sets this element’s color.

Line 34

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 35

```
Text(
```

Draws text using the supplied value/style.

Line 36

```
text = "${history.size} ENTRIES",
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 37

```
style = MaterialTheme.typography.labelSmall,
```

Chooses a theme text style or a customized copy of it.

Line 38

```
color = Color.Gray
```

Sets this element’s color.

Line 39

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 40

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 41

```
Spacer(modifier = Modifier.height(8.dp))
```

Adds empty space between UI elements.

Line 42

```
HorizontalDivider(color = Color.Black, thickness = 2.dp)
```

Draws a dividing line.

Line 43

```
Spacer(modifier = Modifier.height(16.dp))
```

Adds empty space between UI elements.

Line 44

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 45

```
if (history.isEmpty()) {
```

Shows the empty-history message when there are no saved rows.

Line 46

```
Box(modifier = Modifier.fillMaxSize(), contentAlignment =
 Alignment.Center) {
```

A container that can position/stack its children.

Line 47

```
Text("No saved searches yet.", style =
MaterialTheme.typography.bodyMedium, color = Color.Gray)
```

Draws text using the supplied value/style.

Line 48

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 49

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 50

```
LazyColumn(modifier = Modifier.fillMaxSize(),
verticalArrangement = Arrangement.spacedBy(8.dp)) {
```

Draws a scrollable vertical list that creates items as needed.

Line 51

```
items(history) { item ->
```

Draws one history row per saved item; the list has already been ordered by Room.

Line 52

```
Surface(
```

A visual container providing background, border and shape.

Line 53

```
modifier = Modifier
```

Applies the following layout/appearance instructions to this UI element.

Line 54

```
.fillMaxWidth()
```

Uses the maximum available width in the parent layout.

Line 55

```
.clickable { onSelectWord(item.word) },
```

Tells App which saved word was tapped, so it can open that result.

Line 56

```
color = if (item.isFavorite) Color(0xFFFFF9C4) else
Color.White,
```

Only displays the extra star badge for favorite items.

Line 57

```
border = BorderStroke(1.dp, Color.Black),
```

Sets the outline width and color.

Line 58

```
shape = Shapes().extraSmall
```

Chooses the component’s corner shape.

Line 59

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 60

```
Row(
```

A horizontal layout: the children below are arranged side by side.

Line 61

```
modifier = Modifier.padding(16.dp).fillMaxWidth(),
```

Applies the following layout/appearance instructions to this UI element.

Line 62

```
horizontalArrangement = Arrangement.SpaceBetween,
```

Controls how children are spread horizontally.

Line 63

```
verticalAlignment = Alignment.CenterVertically
```

Controls vertical alignment of children.

Line 64

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 65

```
Column(modifier = Modifier.weight(1f)) {
```

A vertical layout: the children below are arranged top to bottom.

Line 66

```
Row(verticalAlignment = Alignment.CenterVertically) {
```

A horizontal layout: the children below are arranged side by side.

Line 67

```
Text(
```

Draws text using the supplied value/style.

Line 68

```
text = item.word.uppercase(),
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 69

```
style =
MaterialTheme.typography.titleMedium.copy(fontWeight =
FontWeight.Bold),
```

Chooses a theme text style or a customized copy of it.

Line 70

```
color = Color.Black
```

Sets this element’s color.

Line 71

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 72

```
if (item.isFavorite) {
```

Only displays the extra star badge for favorite items.

Line 73

```
Spacer(modifier = Modifier.width(8.dp))
```

Adds empty space between UI elements.

Line 74

```
Text("★", color = Color(0xFFF57F17))
```

Draws text using the supplied value/style.

Line 75

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 76

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 77

```
Spacer(modifier = Modifier.height(4.dp))
```

Adds empty space between UI elements.

Line 78

```
Text(
```

Draws text using the supplied value/style.

Line 79

```
text = item.snippet,
```

Provides the text to display; interpolated $ values are evaluated from the current data.

Line 80

```
style = MaterialTheme.typography.bodySmall,
```

Chooses a theme text style or a customized copy of it.

Line 81

```
color = Color.DarkGray,
```

Sets this element's color.

Line 82

```
maxLines = 1
```

Limits how many lines this text can occupy.

Line 83

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 84

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 85

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 86

```
IconButton(onClick = { onToggleFavorite(item.word) }) {
```

Tells App which word's star should be changed. The database update occurs outside this UI file.

Line 87

```
Text(if (item.isFavorite) "★" else "☆", style =
MaterialTheme.typography.titleLarge)
```

Only displays the extra star badge for favorite items.

Line 88

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 89

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 90

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 91

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 92

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 93

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 94

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 95

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 96

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.